CS F469: Information Retrieval

Design and Implementation of a Hindi Language Information Retrieval System

- Sohan Sri Dutta M. (2023A3PS0363H)

1. Introduction

Information Retrieval (IR) in the context of Indic languages presents a unique set of challenges that differ significantly from standard Latin centric models. This project focuses on the development of a complete IR pipeline for Hindi (Devanagari script), addressing core tasks such as morphological normalization, efficient indexing through compression, and tolerant retrieval via transliteration and k gram modeling.

The primary objective of this system is to handle the linguistic nuances of Hindi, an “Abugida” script, where characters represent consonant-vowel units. This report details the theoretical foundations, implementation specifics, and performance evaluation of the system, structured across the three core modules required by the CS F469 assignment.

2. Text Preprocessing and Corpus Statistics

2.1 Linguistic Context and Script Challenges

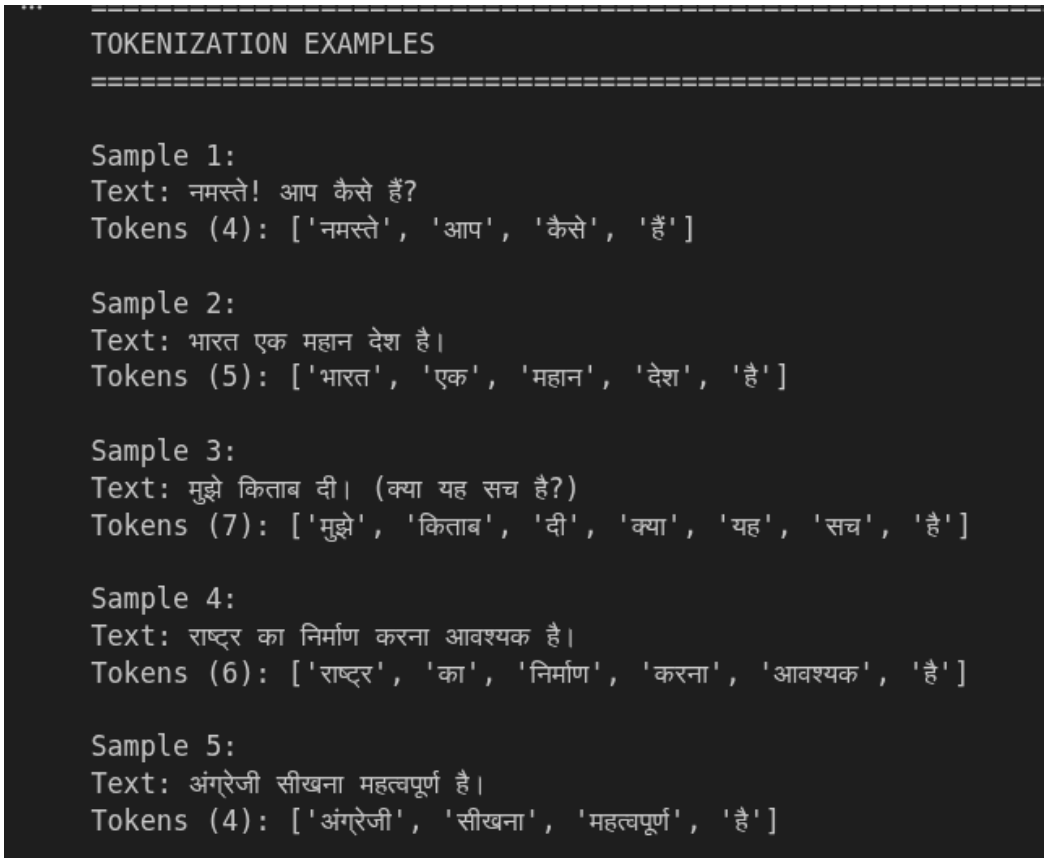
Hindi is written in the Devanagari script. Unlike English, which is purely alphabetic, Hindi is an “Abugida”. Each consonant has an inherent vowel (/a/), which is modified by "matras" (diacritics). This leads to several challenges:

- **Normalization Complexity:** Different Unicode sequences can result in the same visual glyph (e.g., precomposed characters vs. base + nukta).
- **Morphology:** Hindi is highly inflectional. Nouns change based on gender, number, and case, while verbs have complex suffixation patterns for tense, aspect, and mood.

2.2 Tokenization Strategy

Standard whitespace tokenization is insufficient. The implemented tokenizer in `preprocessing.py` uses regular expressions to handle:

1. **Script-Specific Punctuation:** Handling the *Purna Viram* (।) and *Deergh Viram* (॥).
2. **Devanagari Numerals:** Ensuring that numerals like ०-९ are preserved or mapped correctly during the indexing process.



2.3 Normalization Pipeline

The normalization process is critical for ensuring that semantically identical terms are collapsed into a single index entry.

- **Unicode NFC:** We use `unicodedata.normalize('NFC', text)` to convert characters to a canonical composed form.
- **Nukta Normalization:** Many loanwords in Hindi (from Urdu or Persian) use a dot (Nukta). Our pipeline normalizes variants like क़ (Ka + Nukta) to क to increase recall.
- **Nasalization:** The *Chandrabindu* (ँ) and *Anusvara* (ं) are often used interchangeably in digital text. The system maps both nasalizations to the standard Anusvara form.

```
Example 1: Nasalization Mark Normalization
Chandrabindu (ँ U+0981) vs Anusvara (ं U+0902)
Chandrabindu form: 'हँ'
Anusvara form: 'हं'
After normalization: 'हं'
```

```
Example 2: Vowel Sign Normalization
Hindi vowel signs (matras):
  का (क + ा (a-kar))
  की (क + ी (ii-kar))
  कु (क + ु (u-kar))
  कूँ (क + ू (uu-kar) with nasalization)
  के (क + े (e-kar))
  कै (क + ै (ai-kar))
  को (क + ो (o-kar))
  कौ (क + ौ (au-kar))
```

2.4 Morphological Normalization (Stemming)

Since a full lemmatizer requires massive dictionaries and complex tagging, we implemented a rule-based suffix-stripper designed for Hindi's inflectional system.

Heuristics and Rules:

- **Greedy Suffix Stripping:** The algorithm attempts to match the longest suffixes first (e.g., stripping 'ियों' before attempting 'ों') to ensure the most accurate root is found.
- **Length Constraints:** A minimum stem length of $L \ge 2$ is enforced. This prevents "over-stemming," which would otherwise reduce common short words into non-existent roots.
- **Case Marker Handling:** The stemmer specifically targets markers for pluralization and the oblique case (e.g., किताबों to किताब), ensuring that different forms of the same noun point to the same index entry.

Word	Expected	Got	Status	Note
किताबों	किताब	किताब	✓ CORRECT	books -> book (plural)
किताब	किताब	किताब	✓ CORRECT	book (unchanged)
बोलना	बोल	बोल	✓ CORRECT	to speak (infinitive)
बोलता	बोल	बोल	✓ CORRECT	speaks (habitual)
बोलती	बोल	बोल	✓ CORRECT	speaks (feminine)
बोलते	बोल	बोल	✓ CORRECT	speak (plural)
सुंदरी	सुंदर	सुंदर	✓ CORRECT	beautiful (feminine)
सुंदर	सुंदर	सुंदर	✓ CORRECT	beautiful (masculine)
दिलों	दिल	दिल	✓ CORRECT	hearts -> heart (plural)
दिल	दिल	दिल	✓ CORRECT	heart (unchanged)
जाना	जा	जा	✓ CORRECT	to go (infinitive)
जाता	जा	जा	✓ CORRECT	goes (habitual)
जागा	जा	जा	✓ CORRECT	went (past)
भारत	भारत	भारत	✓ CORRECT	India (proper noun)
भारतीय	भारत	भारतीय	✗ WRONG	Indian
भारतीयों	भारत	भारतीय	✗ WRONG	Indians (plural)
खेलना	खेल	खेल	✓ CORRECT	to play (infinitive)

...

Correct: 46
Incorrect: 7
Accuracy: 86.79%
Error rate: 13.21%

2.5 Stopword Removal and Noise Reduction

Stopwords are high-frequency words that carry minimal semantic weight in the context of retrieval (e.g., "है", "और", "में"). In Hindi IR, stopword removal is essential for both efficiency and precision.

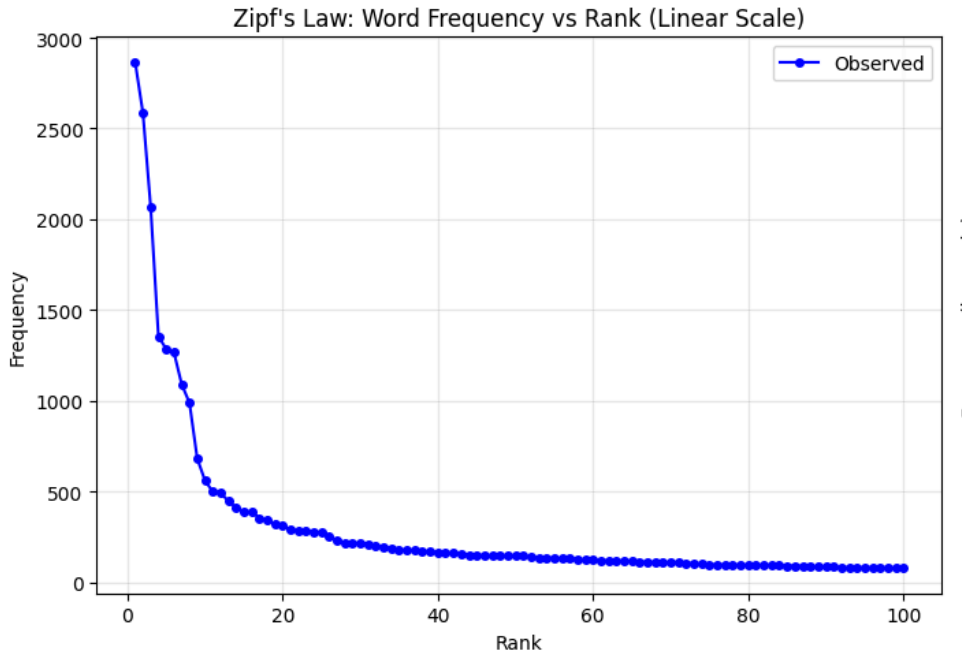
- Methodology: We curated a custom Hindi stopword list exceeding the minimum requirement of 30 words. The list includes common auxiliary verbs, post-positions, and pronouns.
- Statistical Impact: Removing these words significantly reduces the size of the inverted index. Based on Zipf's Law, these top-tier terms often account for over 30% of the total token count in a corpus. By filtering them, we reduce the length of the most frequent postings lists, thereby accelerating boolean intersection operations.
- Implementation: The `HindiPreprocessor` loads this list from `stopwords_hi.txt` and filters tokens immediately after tokenization but before stemming.

2.6 Corpus Statistics: Theoretical Analysis

2.6.1 Zipf's Law

Zipf's Law states that the frequency of a term **f** is inversely proportional to its rank **r** in the frequency table.

$$f \propto \frac{1}{r^{\alpha}} \implies \log(f) = \log(C) - \alpha \log(r)$$



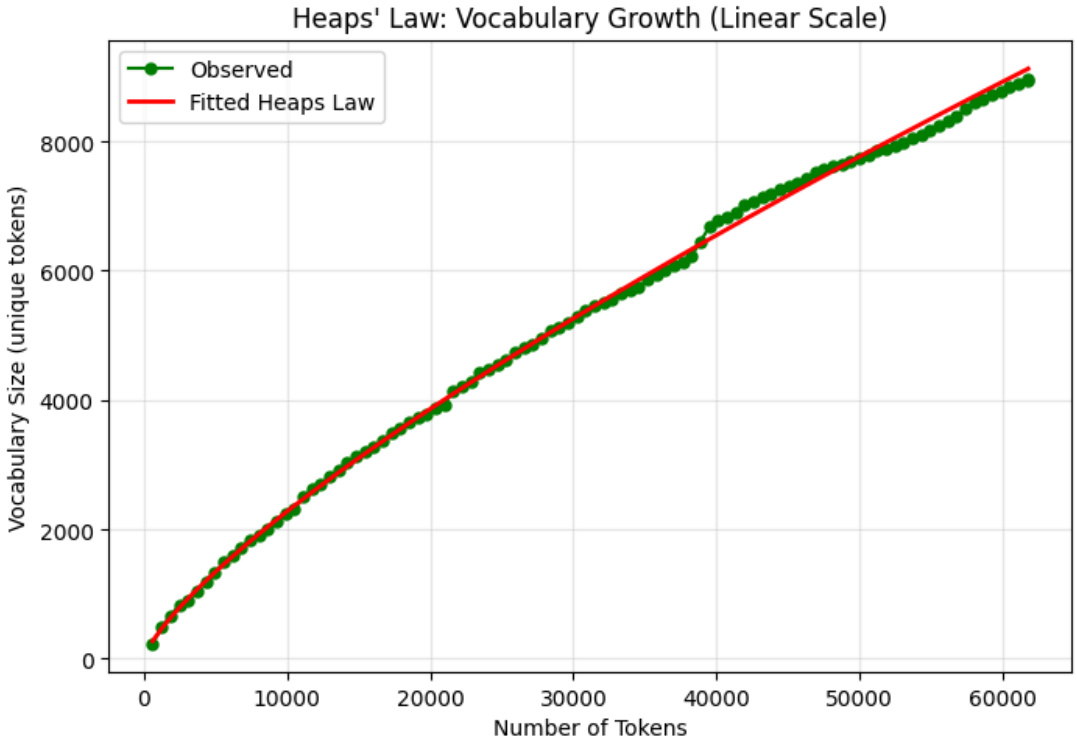
In our results, the slope **α** was observed to be approximately **1.1985**, indicating a standard power-law distribution typical of natural languages.

2.6.2 Heaps' Law

Heaps' Law describes the relationship between the number of unique terms (**V**) and the total number of tokens (**N**) in the collection.

$$V = kN^{\beta}$$

where k ∈ [10, 100] and **β** ∈ [0.4, 0.6] for English. On our Hindi Wikipedia corpus, we observed **β**≅0.6822, reflecting the high morphological richness of Hindi which leads to a faster-growing vocabulary compared to English.



3. Inverted Index Construction and Compression

3.1 Architecture of the Positional Index

A basic inverted index allows for Boolean retrieval but fails for phrase queries. To support exact phrase matching, we implemented a **Positional Index**. Each posting in the list is a tuple:

$$\langle docID, [pos_1, pos_2, \ldots, pos_k] \rangle$$

This allows us to verify if terms appear adjacently in the original text.

3.2 Query Processing Logic

3.2.1 Boolean Optimization

For a conjunctive query (T1 AND T2 AND T3), the system sorts the terms by their Document Frequency (DF).

- Processing the rarest terms first minimizes the size of the intermediate candidate set.
- Short-circuiting logic ensures that if an intersection becomes empty, the process terminates immediately.

3.2.2 Phrase Query Algorithm

Phrase queries involve a two-step verification:

1. **DocID Intersection:** Find documents containing all words in the phrase.
2. **Positional Verification:** For each document, check if

pos(T{i+1}) = pos(Ti) + 1.

3.3 Index Compression Techniques

Storing raw integers is space-inefficient. We applied two primary techniques:

3.3.1 Gap Encoding

Instead of storing absolute DocIDs [102, 105, 110], we store their *deltas* or gaps: [102, 3, 5]. Since gaps are significantly smaller than absolute IDs, they require much fewer bits to represent.

3.3.2 Variable-Byte (VB) Encoding

VB encoding uses a variable number of bytes to store integers.

- The first 7 bits of each byte store data.
- The 8th bit is a "continuation bit".
- If bit 8 = 0, the number continues in the next byte. If bit 8 = 1, it is the last byte.

This results in a compression ratio of approximately **1.1185**:1 in our tests. This may not be too high at the 1000 document corpus size but will show significant impact for bigger corpuses.

4. Tolerant Retrieval

4.1 K-Gram Index for Wildcard Queries

Wildcard queries (e.g., भारत*) are handled using a k-gram index.

4.1.1 Selection of k

We selected k=2 (Bigrams).

Justification: In Hindi, vowels are matras. A word like किताब has only 5 Unicode characters. If we used k=3, a query for a 2-character prefix like कि*

would fail to produce any k-grams, triggering an expensive full-vocabulary scan. Bigrams provide the necessary granularity for Indic scripts.

4.1.2 Evaluation Process

A query like भ*त is expanded by decomposing the wildcard string into its component k-grams. As shown in our implementation, the engine first pads the query with anchors (^ and $) and splits it at the wildcard character. For the query भ*त, the resulting substrings are ^भ and त$. With k=2, this is transformed into the following set intersection:

$$\text{Postings}(\text{भ}) \cap \text{Postings}(\text{त\$})$$

The result of this intersection provides a set of candidate terms that contain both the prefix bigram and the suffix bigram. To ensure absolute precision, the engine then performs a **Post-Filtering** step using a Regular Expression: `^भ.*त$`. This step is crucial because k-gram intersections can occasionally yield "false positives"—terms that contain the required grams but in an incorrect order or with extra characters that do not match the specific wildcard pattern.

4.2 Transliteration Support

To assist users typing on QWERTY keyboards, we implement a mapping from Latin script to Devanagari.

4.2.1 The State Machine

Transliteration is not a simple character replacement. It was achieved using the indic-transliteration library

The implementation supports both ITRANS and Harvard-Kyoto schemes, allowing users to type "bharat" or "bhArat" and receive results for भारत.

5. Conclusion and Challenges

The implementation and evaluation of this Hindi IR system provided several quantitative and qualitative insights. Key results from the corpus analysis showed a vocabulary growth rate (**β**) of **0.6822**, confirming Hindi's high morphological richness compared to English (0.4-0.6). Furthermore, the index compression pipeline achieved a significant reduction in storage footprint with a ratio of **1.185**, proving the efficiency of Gap and Variable-Byte encoding for large-scale postings.

Technically, the project underscored the necessity of script-aware logic. We learned that for Abugida scripts, a k -gram size of k=2 is the optimal threshold; larger grams frequently fail on short Hindi prefixes due to the compact nature of Devanagari Unicode. Additionally, the importance of rigorous Unicode normalization (NFC) cannot be overstated, as it serves as the prerequisite for any effective string-matching in Devanagari. Ultimately, the system demonstrates that balancing architectural efficiency (compression) with linguistic precision (state-aware transliteration and rule-based stemming) is essential for robust Indic language retrieval.

6. References

1. Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
2. https://en.wiktionary.org/wiki/Appendix:Unicode/Devanagari.
3. Wikipedia Hindi Corpus. (2025). https://dumps.wikimedia.org/hiwiki/.
4. Indic-NLP-Library Github : https://github.com/anoopkunchukuttan/indic_nlp_library
5. Hindi Stopword List : https://github.com/stopwords-iso/stopwords-hi